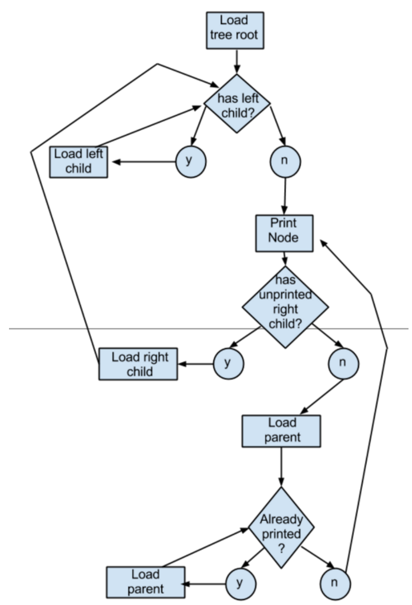
Tree sort

A **tree sort** is a sort algorithm that builds a binary search tree from the elements to be sorted, and then traverses the tree (in-order) so that the elements come out in sorted order.[1] Its typical use is sorting elements online: after each insertion, the set of elements seen so far is available in sorted order.

Tree sort can be used as a one-time sort, but it is equivalent to quicksort as both recursively partition the elements based on a pivot, and since quicksort is in-place and has lower overhead, tree sort has few advantages over quicksort. It has better worst case complexity when a self-balancing tree is used, but even more overhead.

Tree sort



Class	Sorting algorithm
Data structure	Array
Worst-case performance	$O(n^2)$ (unbalanced) $O(n \log n)$ (balanced)
Best-case performance	$O(n \log n)$
Average performance	$O(n \log n)$
Worst-case space complexity	$\Theta(n)$
Optimal	Yes, if balanced

Efficiency

Adding one item to a binary search tree is on average an $O(\log n)$ process (in big O notation). Adding n items is an $O(n \log n)$ process, making tree sorting a 'fast sort' process. Adding an item to an unbalanced binary tree requires $O(n)$ time in the worst-case: When the tree resembles a linked list (degenerate tree). This results in a worst case of $O(n^2)$ time for this sorting algorithm. This worst case occurs when the algorithm operates on an already sorted set, or one that is nearly sorted, reversed or nearly reversed. Expected $O(n \log n)$ time can however be achieved by shuffling the array, but this does not help for equal items.

The worst-case behaviour can be improved by using a self-balancing binary search tree. Using such a tree, the algorithm has an $O(n \log n)$ worst-case performance, thus being degree-optimal for a comparison sort. However, tree sort algorithms require separate memory to be allocated for the tree, as opposed to in-place algorithms such as quicksort or heapsort. On most common platforms, this means that heap memory has to be used, which is a significant performance hit when compared to quicksort and heapsort. When using a splay tree as the binary search tree, the resulting algorithm (called splaysort) has the additional property that it is an adaptive sort, meaning that its running time is faster than $O(n \log n)$ for inputs that are nearly sorted.

Example

The following tree sort algorithm in pseudocode accepts a collection of comparable items and outputs the items in ascending order:

```
STRUCTURE BinaryTree
    BinaryTree:LeftSubTree
    Object:Node
    BinaryTree:RightSubTree

PROCEDURE Insert(BinaryTree:searchTree, Object:item)
    IF searchTree.Node IS NULL THEN
        SET searchTree.Node TO item
    ELSE
        IF item IS LESS THAN searchTree.Node THEN
            Insert(searchTree.LeftSubTree, item)
        ELSE
            Insert(searchTree.RightSubTree, item)

PROCEDURE InOrder(BinaryTree:searchTree)
    IF searchTree.Node IS NULL THEN
        EXIT PROCEDURE
    ELSE
        InOrder(searchTree.LeftSubTree)
        EMIT searchTree.Node
        InOrder(searchTree.RightSubTree)

PROCEDURE TreeSort(Collection:items)
    BinaryTree:searchTree

    FOR EACH individualItem IN items
        Insert(searchTree, individualItem)

    InOrder(searchTree)
```

In a simple functional programming form, the algorithm (in Haskell) would look something like this:

```haskell
data Tree a = Leaf | Node (Tree a) a (Tree a)

insert :: Ord a => a -> Tree a -> Tree a
insert x Leaf = Node Leaf x Leaf
insert x (Node t y s)
    | x <= y = Node (insert x t) y s
    | x > y  = Node t y (insert x s)

flatten :: Tree a -> [a]
flatten Leaf = []
flatten (Node t x s) = flatten t ++ [x] ++ flatten s

treesort :: Ord a => [a] -> [a]
treesort = flatten . foldr insert Leaf
```

In the above implementation, both the insertion algorithm and the retrieval algorithm have $O(n^2)$ worst-case scenarios.

External links

- Binary Tree Java Applet and Explanation (https://web.archive.org/web/20161129234513/http://qmatica.com/DataStructures/Trees/BST.html) at the Wayback Machine (archived 29 November 2016)
- Tree Sort of a Linked List (https://web.archive.org/web/20160810130303/https://www.martinbroadhurst.com/articles/sorting-a-linked-list-by-turning-it-into-a-binary-tree.html) at the Wayback Machine (archived 10 August 2016)
- Tree Sort in C++ (https://web.archive.org/web/20220121202457/https://www.martinbroadhurst.com/cpp-sorting.html) at the Wayback Machine (archived 21 January 2022)

References

1. McLuckie, Keith; Barber, Angus (1986). "Binary Tree Sort". *Sorting routines for microcomputers*. Basingstoke: Macmillan. ISBN 0-333-39587-5. OCLC 12751343 (https://search.worldcat.org/oclc/12751343).